

Taller · Tu app en una VM real de Azure — de cero a Docker Hub

Curso 101-EAN · Gestión de Contenedores · Sesión 3 (taller complementario) Duración estimada: 45–60 min · Cuenta Azure for Students + cuenta Docker Hub

Hasta ahora practicaste en Play with Docker. Hoy lo haces **en infraestructura real**: creas tu propia VM en Azure, le instalas Docker, construyes una app de login servida por nginx, la publicas en el puerto 80 al mundo (abriendo la puerta del firewall tú mismo) y subes la imagen a Docker Hub.

0. Requisitos

- Cuenta **Azure for Students** activa (azure.microsoft.com/free/students) — \$100 de crédito, sin tarjeta.
- Tu cuenta de **Docker Hub** (usuario y contraseña a la mano).

💰 **Costo del taller**: la VM que usaremos (`Standard_B2als_v2`) cuesta ~US\$0.03/hora. El taller completo consume **menos de \$0.10** de tu crédito — si al final la **detienes (deallocate)**. No la dejes corriendo por días.

1. Crea tu máquina virtual (10 min)

En portal.azure.com → **Virtual machines** → **Create** → **Azure virtual machine**, con estos valores (los demás se dejan por defecto):

Campo	Valor
Resource group	Create new → EAN
Virtual machine name	eanlab
Region	Mexico Central (si no aparece: East US)
Availability options	Availability zone (deja la zona sugerida)
Security type	Trusted launch (por defecto)
Image	Ubuntu Server 24.04 LTS – x64 Gen2
Size	Standard_B2als_v2 (2 vcpus, 4 GiB) — clic en <i>See all sizes</i> si no aparece
Authentication type	Password
Username	azureuser
Password	una contraseña FUERTE (mínimo 12 caracteres, mayúsculas, números, símbolo)
Public inbound ports	Allow selected ports → SSH (22)
Disks → OS disk size	30 GiB (por defecto)

Review + create → **Create**. En ~2 minutos está lista.

 La contraseña de la VM es una credencial real en una IP pública: fuerte y solo tuya. (Los bots escanean el puerto 22 de TODO Azure a los minutos de crearse una IP.)

Conéctate por SSH (Terminal en Mac/Linux, PowerShell en Windows). La IP pública está en el Overview de la VM:

```
ssh azureuser@IP-PUBLICA-DE-TU-VM
```

2. Instala Docker en la VM (5 min)

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker
sudo systemctl enable --now docker

docker run hello-world      # ← debe saludar (ojo: hello-world, con L final)
```

3. La app de login (5 min)

```
mkdir ~/login-app && cd ~/login-app
```

Crea el archivo `index.html` pegando este bloque **completo** en la terminal (crea el archivo de una sola vez, sin editor):

```

cat > index.html <<'EOF'
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Portal Demo · 101-EAN</title>
<style>
  body { margin:0; font-family:'Segoe UI',sans-serif; min-height:100vh; display:flex;
    align-items:center; justify-content:center;
    background:linear-gradient(135deg,#00A651,#003DA5); }
  .card { background:#fff; padding:40px 36px; border-radius:14px; width:320px;
    box-shadow:0 20px 50px rgba(0,0,0,.3); }
  h1 { margin:0 0 6px; color:#0D1B4B; font-size:22px; }
  p.sub { margin:0 0 24px; color:#5A646E; font-size:13px; }
  label { display:block; font-size:12px; color:#0D1B4B; font-weight:600; margin-bottom:4px; }
  input { width:100%; padding:10px 12px; margin-bottom:16px; border:1px solid #D5DCE3;
    border-radius:8px; font-size:14px; box-sizing:border-box; }
  button { width:100%; padding:12px; background:#00A651; color:#fff; border:none;
    border-radius:8px; font-size:15px; font-weight:600; cursor:pointer; }
  button:hover { background:#008f45; }
  .foot { margin-top:18px; text-align:center; font-size:11px; color:#5A646E; }
  #msg { color:#00A651; font-weight:600; text-align:center; margin-top:10px; display:none; }
</style>
</head>
<body>
  <div class="card">
    <h1>🐳 Portal Demo – TU NOMBRE</h1>
    <p class="sub">Gestión de Contenedores · Universidad EAN</p>
    <form onsubmit="event.preventDefault(); document.getElementById('msg').style.display='block';">
      <label>Usuario</label>
      <input type="text" placeholder="usuario@ejemplo.com" required>
      <label>Contraseña</label>
      <input type="password" placeholder="....." required>
      <button type="submit">Ingresar</button>
      <div id="msg">✅ Demo: formulario capturado (sin backend)</div>
    </form>
    <div class="foot">Servido por nginx dentro de un contenedor Docker</div>
  </div>
</body>
</html>
EOF

```

Personalízalo: cambia **TU NOMBRE** por el tuyo — es TU app. Para editarlo: [vi index.html](#) (o vuelve a pegar el bloque con tus cambios).

(El archivo también está en esta carpeta del repo — `index.html` — por si prefieres clonarlo: `git clone https://github.com/Hackwy402/gestion_de_contenedores.git`)

⚠ Es una **maqueta sin backend**: el formulario no envía datos a ningún lado. Nunca publiques un formulario real de credenciales sin cifrado ni backend seguro.

Y el Dockerfile (también está en `Dockerfile`):

```
cat > Dockerfile <<'EOF'
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
EXPOSE 80
EOF
```

4. Construye y despliega en el puerto 80 (5 min)

```
docker build -t login-app:1.0 .
docker run -d --name portal -p 80:80 login-app:1.0
curl -s localhost | head -5      # debe devolver tu HTML
```

Si el 80 está ocupado (`address already in use`): mira quién es con `sudo ss -tlnp | grep :80` — si es un nginx del sistema: `sudo systemctl disable --now nginx` y repite el `docker run` (antes `docker rm -f portal`).

5. Abre la puerta del firewall (NSG) (5 min)

Tu contenedor escucha, pero Azure bloquea el 80 por defecto — **tú decides qué se expone** (la lección de la S3, en la vida real):

Portal → tu VM → **Networking** → **Network settings** → + **Create port rule** → **Inbound port rule**:

- Destination port ranges: `80` · Protocol: **TCP** · Priority: `310` · Name: `Allow-HTTP-80`

Add, espera ~30 segundos y abre en tu navegador:

```
http://IP-PUBLICA-DE-TU-VM
```

🌐 Tu app de login, servida desde TU VM, visible para el mundo.

6. Publícala en Docker Hub (10 min)

```
docker login # tu usuario del Hub
docker tag login-app:1.0 TU-USUARIO/login-app:1.0
docker push TU-USUARIO/login-app:1.0
```

Verifícala en <https://hub.docker.com/r/TU-USUARIO/login-app>.

La prueba de portabilidad — en [Play with Docker](#) (¡otra máquina, otra nube!):

```
docker run -d -p 8080:80 TU-USUARIO/login-app:1.0
```

Abre el 8080: tu app, construida en Azure, corriendo donde quieras.

7. 💰 Cierre FinOps — no quemes tu crédito (2 min)

```
docker rm -f portal
exit
```

Y en el portal: tu VM → **Stop** (o desde tu equipo: `az vm deallocate -g EAN -n eanlab`). La VM detenida (deallocated) **no cobra cómputo**; el disco cuesta centavos al mes. Para las próximas sesiones solo le das **Start** (ojo: la IP pública puede cambiar).

Checklist del taller

- ☐ Creé mi VM Ubuntu 24.04 en Azure (B2als_v2, grupo EAN).
- ☐ Instalé Docker y corrió hello-world.
- ☐ Construí login-app:1.0 con mi nombre en la página.
- ☐ La vi en http://IP-PUBLICA (abrí el 80 en el NSG yo mismo).
- ☐ Publiqué la imagen en Docker Hub y la corrí en Play with Docker.
- ☐ Detuve (deallocate) mi VM al terminar.

Entregable

- URL de tu imagen** en Docker Hub (`hub.docker.com/r/TU-USUARIO/login-app`).
- Captura de tu app abierta en el navegador **con la IP pública visible**.
- Captura de la VM en estado **Stopped (deallocated)** — la evidencia FinOps. 🙄

Errores comunes

Síntoma	Causa / arreglo
<code>Cannot connect to the Docker daemon</code>	El motor no corre → <code>sudo systemctl enable --now docker</code> .
<code>pull access denied for hello-world</code>	Typo: es hello-world.
<code>permission denied ... docker.sock</code>	Falta el grupo → <code>newgrp docker</code> o reconectar el SSH.
<code>port is already in use</code> en el 80	Otro proceso lo tiene → <code>sudo ss -tlnp \ grep :80</code> y detenerlo.
El navegador da timeout	Falta la regla NSG del puerto 80 (paso 5).
<code>denied</code> al hacer push	El tag debe llevar TU usuario: <code>TU-USUARIO/login-app:1.0</code> .
No aparece Mexico Central o el tamaño B2als_v2	Usa East US y/o <code>B2ats_v2</code> / <code>B1s</code> — el taller es idéntico.